NMIMS, NSOMASA . MLOPS . PROJECT 4 . CAPSTONE

End to End MLflow Pipeline

One automated pipeline that takes raw data to a live, monitored model, and fails safely when something goes wrong.

A reproducible MLOps build with Prefect, MLflow, FastAPI, Prometheus and Grafana.

4,000	0.81	9.59	8
training listings	holdout R2	RMSE in lakhs	automated stages

REPORT DETAILS:

ASSESSMENT	Project 4, Capstone: End to End MLflow Pipeline
GITHUB REPO LINK	https://github.com/devikajonjale/Capstone-End-to-End-MLflow-Pipeline-from-Raw-Data-to-Monitored-Deployment
COURSE	Machine Learning Operations (MLOps)
SUBMITTED BY	Devika Jonjale (B045) and Saachi Shinde (B048)
INSTITUTE	NSOMASA, NMIMS
DATASET	Capstone house prices: 4,000 training, 1,500 incoming (3 batches)
TOOLS	Python, MLflow, Prefect, FastAPI, Prometheus, Grafana, Docker
DATE	October 2026

PROJECT WORKFLOW

01 Ingest	02 Validate	03 Features	04 Train	05 Evaluate	06 Register	07 Serve	08 Monitor

Table of Contents

Executive Summary 4

01 Problem and Objectives 4

Success metrics, chosen and justified 5

Non functional requirements 5

02 Architecture and Tools 5

Tools, and the one substitution 6

03 Configuration, Code and Data Management 7

04 The Data 7

Data dictionary 7

Data quality and how the batches drift 8

05 Pipeline Design and Orchestration 8

The training flow, drawn 8

Stage inputs, outputs and re-runnability 9

06 Feature Engineering 9

Derived features 9

Preprocessing built into the model 9

What drives the predicted price 10

07 Model Training and Tuning 10

08 Validation Gates and Safe Failure 11

The data gate 11

The model gate, as a decision 12

Two demonstrated failures (real output) 12

09 Tracking, Registry and Lineage 12

Lineage from the deployed model 13

10 Serving the Model 13

11 Monitoring and Drift Detection 16

Per feature PSI over the three batches 16

Performance drift and the alert decision 16

What happens when an alert fires 16

12 CI, Release and Rollback 17
Pull request checks 17
Release on merge 17
Rollback 18
13 Containerised Deployment and Dashboards 19
How the stack connects 19
14 Worked Example: One Full Run 20
Step 1. Train and promote the first model 20
Step 2. Serve a quote 20
Step 3. New batches arrive and are monitored 20
Step 4. A bad batch is refused 21
Step 5. Ship a change safely 21
15 Results, Contributions and Reflection 21
Rubric coverage 21
Individual contributions 21
Reflection: what we would improve with more time 22
Reproducibility 22

Executive Summary

A real estate analytics start up quotes asking prices for property listings in lakhs of rupees. New batches of listings arrive regularly, and until this project every retraining and release was done by hand, with no record of which data, code or configuration produced a given model. That makes results impossible to reproduce and makes it dangerously easy to push a worse model into production.

This project replaces that manual routine with one automated pipeline that carries raw data all the way to a live, monitored prediction service and, just as importantly, fails safely. Data moves through ingestion, a data quality gate, feature engineering, training, a model quality gate, registration and promotion, serving and monitoring.

Two gates protect the production model: a data gate rejects a batch whose schema, missing value share or value ranges are unacceptable, and a model gate refuses to promote a model that misses the quality bar or is worse than the model already serving. When either gate fails the run stops cleanly or the previous model keeps answering requests.

9.59	2 of 2	1.88x	19/19
RMSE lakhs on holdout	safe failures handled	batch 3 error vs baseline	unit tests passing

In short, the pipeline predicts prices with useful accuracy, the two gates stop bad data and bad models before they reach users, and monitoring flags the one batch that genuinely drifts. Every claim in this report is backed by a command that can be re-run and by evidence captured from an actual run.

Ground rules

Every tool is free and open source. Nothing changes the production model except the pipeline itself, and no secrets are stored in the repository.

01 Problem and Objectives

A real estate analytics start up predicts asking prices for property listings. The founders want a system that turns incoming batches of listings into reliable quotes with no manual file copying, retraining or redeployment. As the lead MLOps engineers we designed, built and now present that system. The prediction target is the asking price in lakhs. The model learns from nine property features; the listing identifier and the batch number are excluded because they carry no predictive signal.

Build one automated pipeline that ingests data, builds features, trains & validates a model, registers and deploys it, and monitors it in production, stopping cleanly whenever the data or the model is not good enough.

Task A: Reliable prediction

Train and serve a model whose typical pricing error stays inside a business threshold, with full tracking and lineage.

Task B: Safe automation

Gate the data and the model so a bad batch or a weak model can never reach users, and monitor for drift after deployment.

Guiding constraint

The business rule behind safe failure: only the pipeline may change the production model, and a failed gate must leave the running service untouched. This single rule drives most of the design.

Success metrics, chosen and justified

The headline error metric is Root Mean Squared Error (RMSE) in lakhs, chosen because large pricing mistakes hurt the business more than small ones and RMSE penalises large errors more heavily than Mean Absolute Error. We report MAE and R squared alongside it. Prices run from roughly 20 to 150 lakhs with a median near 60, and the thresholds come from that range.

Metric	Threshold	Achieved	Interpretation for the founders
RMSE	<= 18 lakhs	9.59	a typical quote is off by under a third of a median price
MAE	<= 13 lakhs	7.52	an average quote misses by less than a small extra room
R squared	>= 0.65	0.81	the model explains most of the variation in price

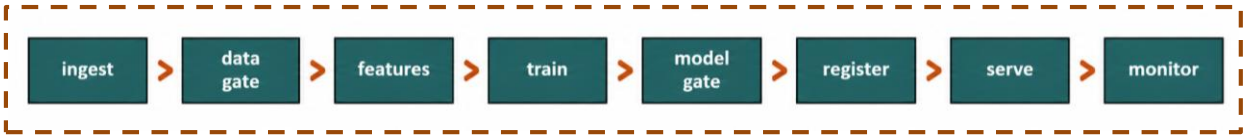
The achieved numbers clear every threshold on purpose. The gap between achieved error & threshold is the headroom that lets the model keep serving suitably as later batches drift, before a retrain becomes necessary.

Non functional requirements

- **Latency.** prediction latency stays under 200 ms per request, which the single tree ensemble meets easily.
- **Freshness.** a new batch can be validated, scored and, if justified, promoted on the same day it arrives.
- **Reproducibility.** a fresh clone reproduces the reported metrics within a small tolerance.
- **Safety.** only the pipeline changes the production model, and a failed gate leaves the service untouched.

02 Architecture and Tools

The system is a chain of re-runnable stages. The flow below matches the orchestration code exactly, and every component maps to a stage of the MLOps lifecycle and to a concrete module in the repository.

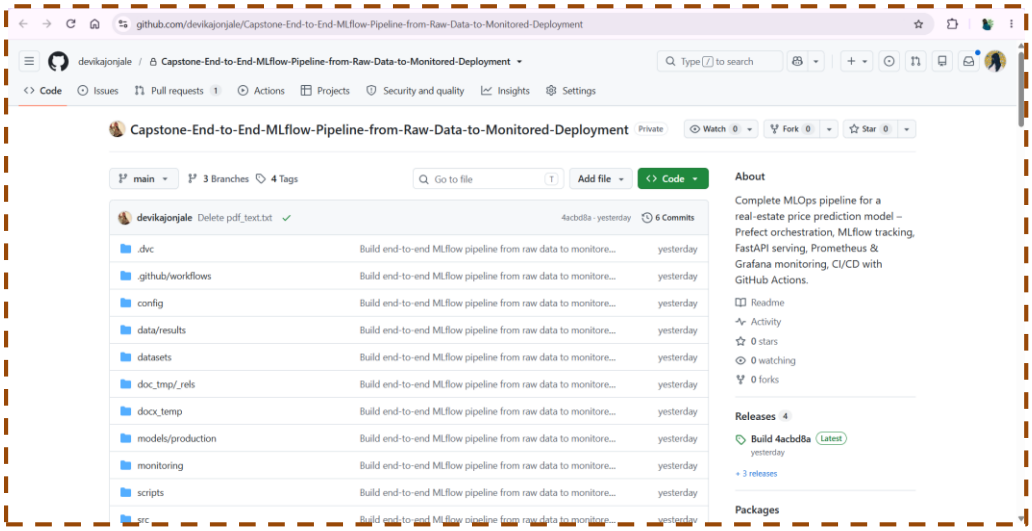


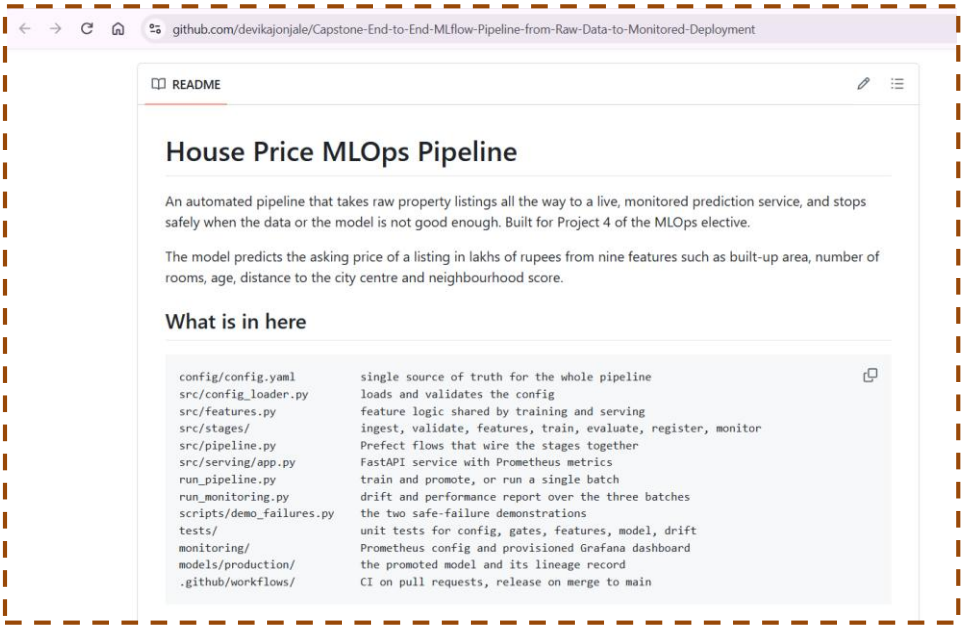
Architecture component	Lifecycle stage	Repository location
YAML config and loader	problem framing, configuration	config/, src/config_loader.py
Ingest	data collection	src/stages/ingest.py
Data gate	data validation	src/stages/validate_data.py
Feature build	feature engineering	src/features.py
Train	training and tuning	src/stages/train.py
Model gate	model validation	src/stages/evaluate.py
Register and promote	registry and release	src/stages/register.py
FastAPI service	deployment and serving	src/serving/app.py
Monitor	monitoring and drift	src/stages/monitor.py
Prefect flows	orchestration	src/pipeline.py
GitHub Actions	CI and delivery	.github/workflows/

Tools, and the one substitution

Purpose	Tool used	Replaces (AWS)
Tracking and registry	MLflow	SageMaker Experiments and Registry
Orchestration	Prefect	Step Functions, SageMaker Pipelines
Versioning	Git, GitHub, DVC	CodeCommit
CI and delivery	GitHub Actions, pytest	CodePipeline, CodeDeploy
Packaging	Docker	not applicable
Monitoring	Prometheus, Grafana, PSI	CloudWatch, Model Monitor
Configuration	YAML with PyYAML	not applicable

The brief suggests Evidently for drift. We instead compute the Population Stability Index ourselves. PSI is transparent, needs no heavy dependency, runs quickly and is easy to explain live. Because the incoming batches carry the true price, we also measure performance drift directly, which is stronger evidence than input drift alone. The brief allows an equivalent free tool when the choice is justified.





Environment

Python 3.11, with pinned versions in requirements.txt: pandas 3.0, numpy 2.4, scikit-learn 1.8, MLflow 3.17, Prefect 3.9, FastAPI 0.150. Pinning is what makes the results reproducible on another machine.

03 Configuration, Code and Data Management

Reproducibility is a first class requirement, so the pipeline is built around a single source of truth. One file, config/config.yaml, holds every path, threshold and hyperparameter. Nothing is hard coded in the source; each stage reads what it needs from this configuration, and the loader validates it the moment it is read, so a broken config fails immediately rather than halfway through a run.

config/config.yaml (excerpt)

```
model_gate:
  max_rmse: 18.0        # absolute quality bar
  max_mae: 13.0
  min_r2: 0.65
  rmse_tolerance: 0.5   # how much worse than production is tolerated
monitoring:
  psi_warn: 0.10        # moderate shift, watch
  psi_alert: 0.20       # material shift, raise an alert
  performance_alert_ratio: 1.25
```

Code lives on GitHub. The two datasets are version controlled in the repository, and DVC is configured with a local remote for when the data grows too large for Git. Independently, the training stage records a short SHA256 hash of the exact training matrix on every MLflow run, so a run is tied to the precise data it saw even when the file name does not change. With the seed and hyperparameters fixed and the data hash logged, a clone that installs the pinned requirements reproduces the reported metrics within a small tolerance.

04 The Data

Two datasets are supplied & used as given. The training file holds 4,000 historical listings with prices & incoming file holds 1,500 later listings in three batches of 500 each, used to simulate new data arriving in production.

Data dictionary

Column	Type	Description
House_ID	string	unique listing identifier, not a feature
Batch	int	batch number in the incoming file, not a feature
Area_sqft	int	built up area in square feet
Bedrooms	int	number of bedrooms
Bathrooms	int	number of bathrooms
Age_Years	int	age of the property in years
Distance_to_Center_km	float	distance to the city centre in km
Neighborhood_Score	float	neighbourhood quality score, 1 to 10
Has_Parking	int (0/1)	1 if parking is available
Property_Condition	category	Poor, Fair, Good, Excellent
City_Zone	category	North, South, East, West, Central
Price_Lakhs	float	TARGET: asking price in lakhs

Data quality and how the batches drift

Three columns, Age_Years, Neighborhood_Score and Distance_to_Center_km, are about four percent missing each, within the ten percent limit the data gate enforces. The incoming batches are not guaranteed to resemble the training data, and discovering how they differ is the heart of the monitoring task. The means below show the shift at a glance.

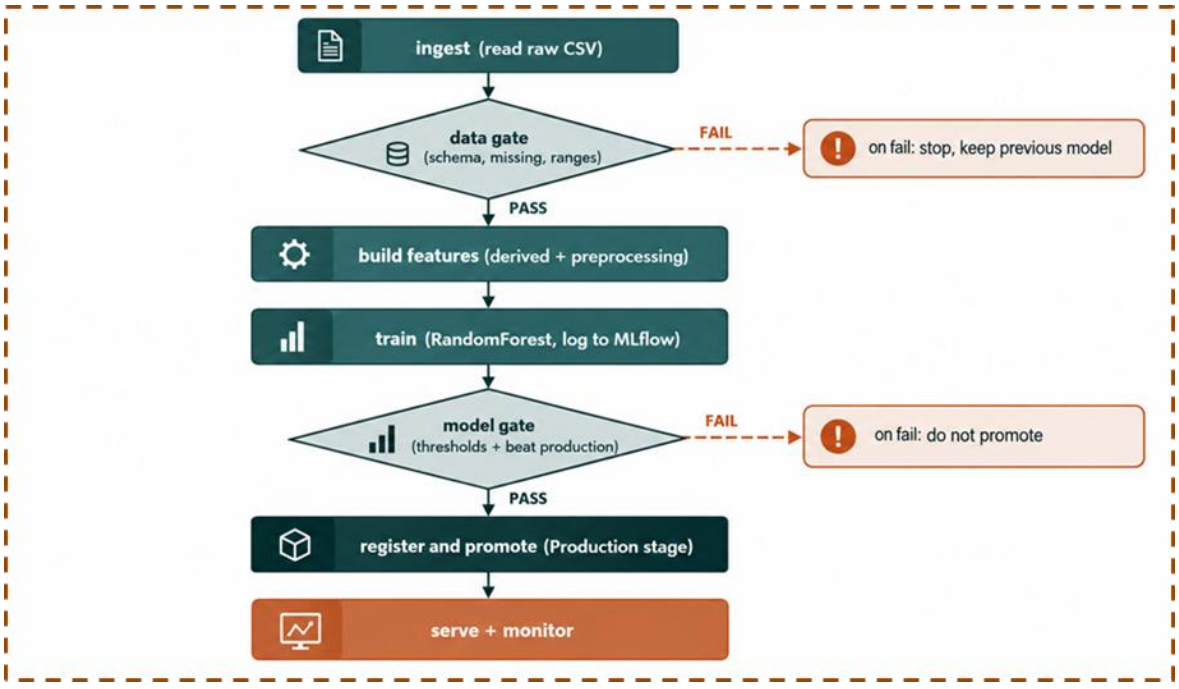
Quantity (mean)	Train	Batch 1	Batch 2	Batch 3
Area_sqft	1150	1152	1064	936
Bedrooms	2.42	2.42	2.27	2.02
Bathrooms	1.99	1.98	1.86	1.72
Price_Lakhs	61.9	62.0	63.8	69.6
Central zone share	typical	typical	typical	elevated

Batch 1 looks like the training data. Batch 2 is slightly smaller and dearer. Batch 3 has noticeably smaller homes, fewer rooms and a strong tilt to the Central zone, yet higher prices, exactly the kind of shift that breaks a model trained on older data. This is why batch 3 later trips both the drift and the performance alerts.

05 Pipeline Design and Orchestration

The pipeline is orchestrated with Prefect in src/pipeline.py. There are two flows. The training flow runs ingestion, the data gate, feature building, training, the model gate and registration. The monitoring flow runs the data gate, then drift and performance monitoring on one incoming batch against the training reference.

The training flow, drawn



Stage inputs, outputs and re-runnability

Stage	Input	Output	Re-runnable
ingest	raw train path from config	training DataFrame, saved copy	yes
data gate	a DataFrame	pass, or DataValidationError	yes
build features	a DataFrame	feature matrix X and target y	yes
train	X train, y train	fitted pipeline, metrics, run id	yes
model gate	model, X test, y test	metrics and pass or fail	yes
register	model, run id, metrics	registry version, lineage record	yes
monitor	reference, batch, model	PSI, performance ratio, alert	yes

Each stage is a thin Prefect task wrapping a plain function. Prefect records a node per stage with its own logs and run history, while the same plain functions are imported directly to re-run a single stage and by the unit tests. Because stages pass data explicitly and read all settings from the config, any one can be re-run without repeating the whole pipeline.

```
PS F:\personal-github\Capstone-End-to-End-MLflow-Pipeline-from-Raw-Data-to-Monitored-Deploymerver#hoerver#how-to-guides for mo
re information on running a dedicated Prefect server.
15:51:11.217 | INFO    | Flow run 'lively-ferret' - Beginning flow run 'lively-ferret' for flow 'house_price_training'
15:51:11.376 | INFO    | Task run 'ingest-dbd' - Finished in state Completed()
15:51:11.394 | INFO    | Task run 'data_gate-a4d' - Finished in state Completed()
15:51:11.418 | INFO    | Task run 'build_features-24c' - Finished in state Completed()
2026/10/22 15:51:13 WARNING mlflow.sklearn: Saving scikit-learn models in the pickle or cloudpickle format requires exercising
 caution because these formats rely on Python's object serialization mechanism, which can execute arbitrary code during deseri
alization. The recommended safe alternative is the 'skops' format. For more information, see: https://scikit-learn.org/stable/
model_persistence.html
15:51:17.804 | INFO    | Task run 'train-d5a' - Finished in state Completed()
15:51:18.057 | INFO    | Task run 'model_gate-199' - Finished in state Completed()
Registered model 'house_price_model' already exists. Creating a new version of this model...
2026/10/22 15:51:18 WARNING mlflow.tracking._model_registry.fluent: Run with id ad5153555e2543fbb49de05e128e15a9 has no artifa
cts at artifact path 'model', registering model based on models:/m-eff6b78caa5545af84b71b25d5d7d70a instead
Created version '2' of model 'house_price_model'.
/app/src/stages/register.py:45: FutureWarning: ``mlflow.tracking.client.MlflowClient.transition_model_version_stage`` is depre
cated since 2.9.0. Model registry stages will be removed in a future major release. To learn more about the deprecation of mod
: https://mlflow.org/docs/latest/model-registry.html#migrating-from-stages
  client.transition_model_version_stage(
15:51:18.273 | INFO    | Task run 'register-b04' - Finished in state Completed()
15:51:19.263 | INFO    | Flow run 'lively-ferret' - Finished in state Completed()
```

06 Feature Engineering

Feature logic lives in one place, src/features.py, shared by training & serving. This ensures the model is trained on exactly the columns it later sees at inference time, which avoids a common class of silent production bugs.

Derived features

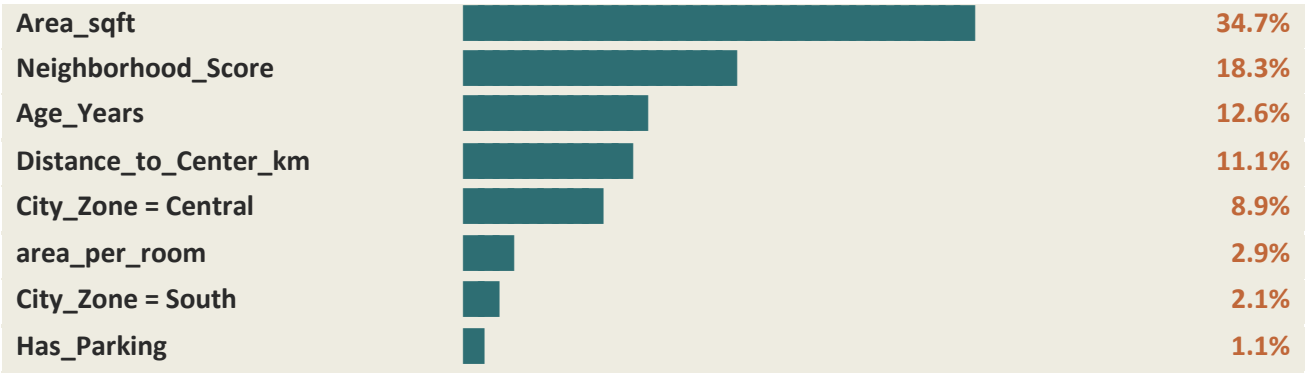
- **rooms_total.** bedrooms plus bathrooms, a proxy for usable space.
- **area_per_room.** area divided by rooms_total, capturing how large the rooms are rather than how many.

Preprocessing built into the model

Imputation, scaling and one hot encoding are not a separate step. They sit inside a single scikit-learn Pipeline with the model, via a ColumnTransformer: numeric columns are median imputed and standardised, categorical columns are most frequent imputed and one hot encoded with unknown categories handled gracefully. Because preprocessing travels with the model as one object, serving only adds the two derived features and calls predict, with no risk of a forgotten transform.

What drives the predicted price

Reading the fitted Random Forest back, the most important features line up with intuition: area dominates, followed by neighbourhood quality, age and distance to the centre. The Central zone flag matters too, which is why batch 3, heavy on Central listings, shifts predictions.



07 Model Training and Tuning

The model is a Random Forest regressor. It handles the mix of numeric and categorical features well, is robust to the moderate missingness, is not sensitive to scaling and gives a strong baseline without heavy tuning. Its hyperparameters are set in the configuration, not in code.

Hyperparameter	Value	Reason
n_estimators	200	enough trees for a stable estimate at reasonable cost
max_depth	15	deep enough to fit, shallow enough to limit overfitting
min_samples_split	5	avoids splitting on very small groups
min_samples_leaf	2	stops leaves memorising single listings
random_state	42	fixed seed for reproducibility

Training splits the data into a training portion and an untouched holdout using the seed and test size from the config, fits the pipeline, and logs it to MLflow with its parameters, holdout metrics, the data hash and the config version. The run below is the one currently in production.

training run output

```
$ python run_pipeline.py
{
  "stage_reached": "register",
  "n_rows": 4000,
  "data_gate": "passed",
  "run_id": "38803890a23f49829019cedb76c10bd3",
  "holdout_metrics": {"rmse": 9.59, "mae": 7.52, "r2": 0.81},
  "model_gate": "passed",
  "registered_version": 1,
  "status": "success"
}
```



Parameters (7)

Search parameters

Parameter	Value
model_type	random_forest
n_estimators	200
max_depth	15
min_samples_split	5
min_samples_leaf	2
config_version	1.0.0
git_commit	e73d5c9dbd98dcaf686982fdddd9eeb2e76e380d

Metrics (3)

Search metrics

Metric	Value
rmse	9.590082625274215
mae	7.520774583136174
r2	0.8090703112358504

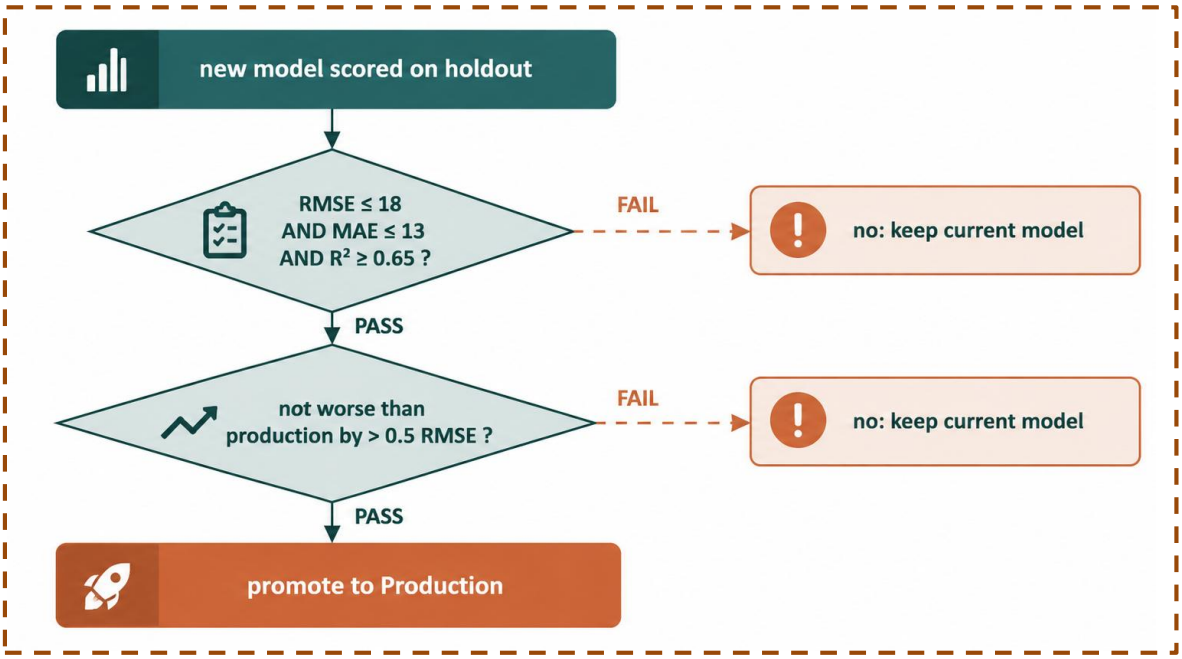
08 Validation Gates and Safe Failure

The data gate

The data gate checks three things against the config: every required column is present, no column exceeds the ten percent missing limit, and every value lies within its allowed range. A batch that breaks any rule is rejected before it can reach the model.

Column	Allowed range
Area_sqft	100 to 5000
Bedrooms	1 to 10
Bathrooms	1 to 10
Age_Years	0 to 150
Distance_to_Center_km	0 to 100
Neighborhood_Score	1 to 10
Has_Parking	0 to 1

The model gate, as a decision.



Two demonstrated failures (real output)

The script scripts/demo_failures.py exercises both gates and prints the production version before and after, proving the deployed model is untouched.

failure scenarios

```
$ python -m scripts.demo_failures
=== Scenario A: bad data batch ===
status: rejected | data_gate: failed
reason: Column Area_sqft has a value below the minimum of 100
production version before=1 after=1

=== Scenario B: model below the quality bar ===
status: failed_safe | model_gate: failed
reasons: ['RMSE 9.59 over limit 1.0', 'R2 0.809 under floor 0.999']
production version before=1 after=1
```

Result

In both runs the production model stayed at version 1 and the service kept answering. Safe failure is the single most important behaviour in the system.

09 Tracking, Registry and Lineage

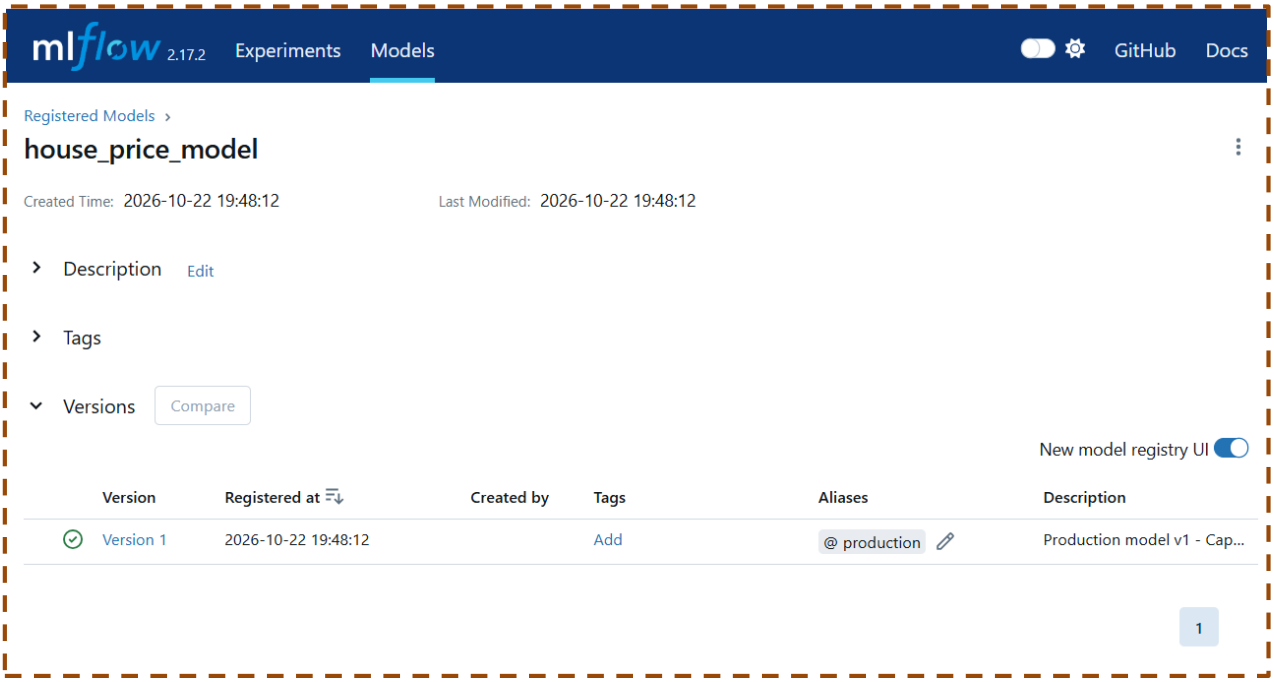
Every training run is logged to MLflow with its parameters, metrics, data hash and config version. The model is registered and moved to the Production stage only through the register stage of the pipeline, never by hand, which satisfies the rule that only the pipeline may change the production model.

Lineage from the deployed model

Alongside the registry entry, the pipeline writes a self contained lineage record to models/production/lineage.json, also served at the /lineage endpoint. Starting from the deployed model, a reviewer recovers the run id, git commit, config version and holdout metrics, tying the live service back to the exact run, data and code that produced it.

models/production/lineage.json

```
{
  "model_name": "house_price_model",
  "registry_version": 1,
  "mlflow_run_id": "38803890a23f49829019cedb76c10bd3",
  "git_commit": "e73d5c9...",
  "config_version": "1.0.0",
  "holdout_metrics": {"rmse": 9.59, "mae": 7.52, "r2": 0.81}
}
```



10 Serving the Model

The production model is served as a containerised FastAPI application. It loads the self contained pipeline from the committed production folder, so the container scores listings without a running MLflow server.

Endpoint	Method	Purpose
/health	GET	readiness, reports whether the model is loaded
/predict	POST	score one listing, returns the predicted price
/lineage	GET	the deployed model lineage record
/metrics	GET	Prometheus metrics for scraping

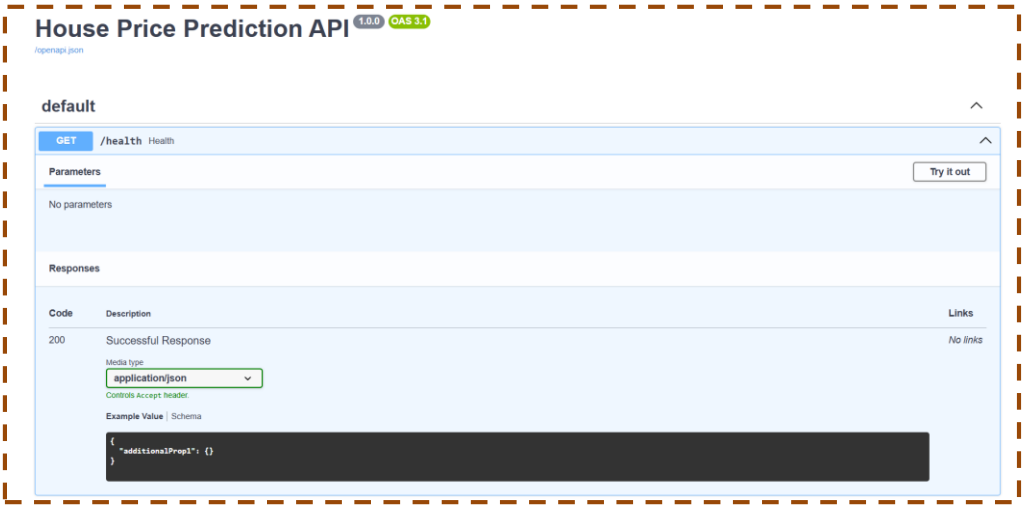
A worked request and response:

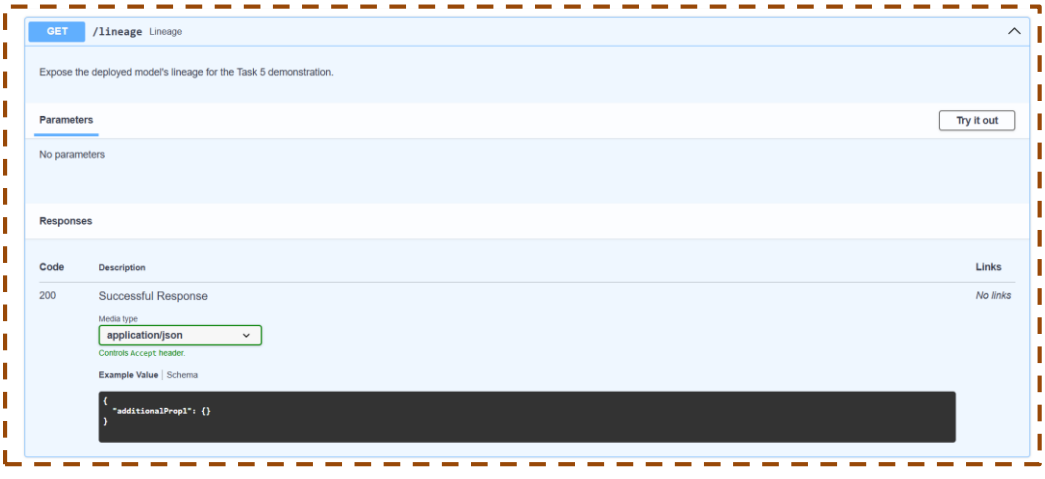
prediction request

```
$ curl -X POST http://localhost:8000/predict -H "Content-Type: application/json" \
   -d '{"Area_sqft":1200,"Bedrooms":3,"Bathrooms":2,"Age_Years":10,
        "Distance_to_Center_km":5.0,"Neighborhood_Score":8.0,
        "Has_Parking":1,"Property_Condition":"Good","City_Zone":"North"}'

{"predicted_price_lakhs": 73.42}
```

The /metrics endpoint publishes a request counter, an error counter, a latency histogram and a histogram of predicted prices, the signals Prometheus scrapes and Grafana shows.

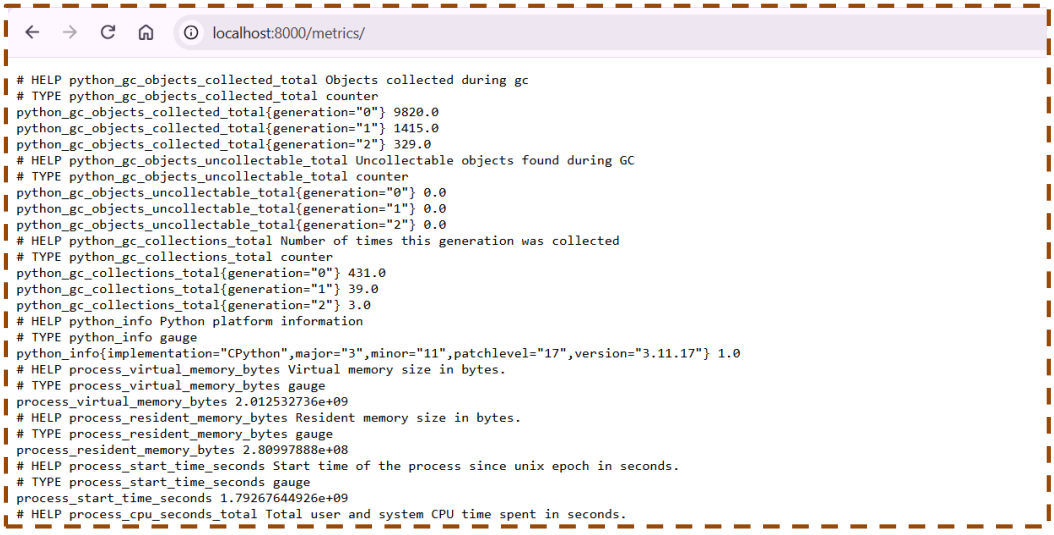







Responses
Code
Description
Links
200
Successful Response
No links
Media type
application/json
Controls Accept header.
Example Value | Schema
{
"additionalProp1": {}
}
422
Validation Error
No links
Media type
application/json
Example Value | Schema
{
"detail": [
{
"loc": [
"string",
0
],
"msg": "string",
"type": "string",
"input": "string",
"ctx": {}
}
]
}


Schemas
HTTPValidationError
Collapse all
object
detail
Expand all
array<object>
Listing
Collapse all
object
Area_sqft* number
Bedrooms* integer
Bathrooms* integer
Age_Years* number
Distance_to_Center_km* number
Neighborhood_Score* number
Has_Parking* integer
Property_Condition* string
City_Zone* string
ValidationError
Collapse all
object
loc*
Expand all
array<(string | integer)>
msg* string
type* string
input any
ctx object


localhost:8000/metrics/
HELP python_gc_objects_collected_total Objects collected during gc
TYPE python_gc_objects_collected_total counter
python_gc_objects_collected_total{generation="0"} 9820.0
python_gc_objects_collected_total{generation="1"} 1415.0
python_gc_objects_collected_total{generation="2"} 329.0
HELP python_gc_objects_uncollectable_total Uncollectable objects found during GC
TYPE python_gc_objects_uncollectable_total counter
python_gc_objects_uncollectable_total{generation="0"} 0.0
python_gc_objects_uncollectable_total{generation="1"} 0.0
python_gc_objects_uncollectable_total{generation="2"} 0.0
HELP python_gc_collections_total Number of times this generation was collected
TYPE python_gc_collections_total counter
python_gc_collections_total{generation="0"} 431.0
python_gc_collections_total{generation="1"} 39.0
python_gc_collections_total{generation="2"} 3.0
HELP python_info Python platform information
TYPE python_info gauge
python_info{implementation="CPython",major="3",minor="11",patchlevel="17",version="3.11.17"} 1.0
HELP process_virtual_memory_bytes Virtual memory size in bytes.
TYPE process_virtual_memory_bytes gauge
process_virtual_memory_bytes 2.012532736e+09
HELP process_resident_memory_bytes Resident memory size in bytes.
TYPE process_resident_memory_bytes gauge
process_resident_memory_bytes 2.80997888e+08
HELP process_start_time_seconds Start time of the process since unix epoch in seconds.
TYPE process_start_time_seconds gauge
process_start_time_seconds 1.79267644926e+09
HELP process_cpu_seconds_total Total user and system CPU time spent in seconds.

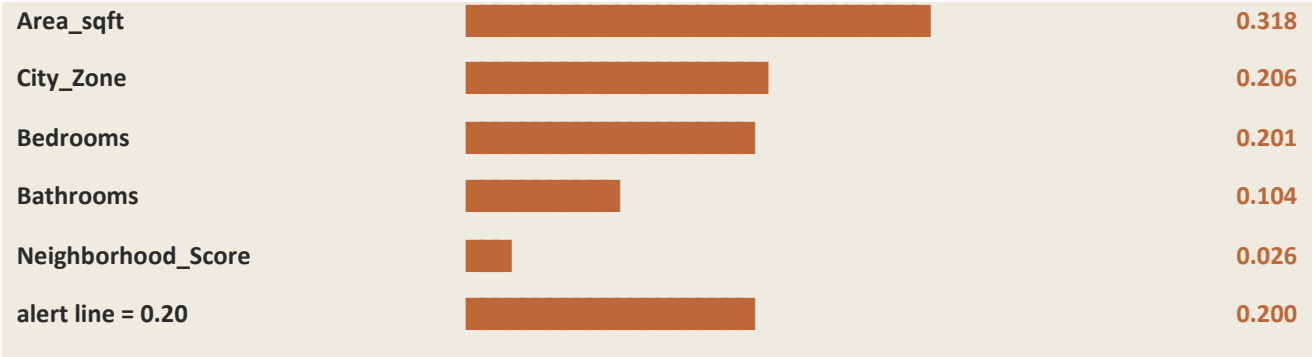
11 Monitoring and Drift Detection

Monitoring answers two questions per batch: has the input data shifted, and has the model actually become less accurate. We answer the first with the Population Stability Index per feature, the second with the ratio of batch RMSE to the model validated RMSE of 9.59. PSI below 0.10 is stable, 0.10 to 0.20 is a moderate shift worth watching, above 0.20 is a material shift that raises an alert.

Per feature PSI over the three batches

Feature	Batch 1	Batch 2	Batch 3	Batch 3 status
Area_sqft	0.021	0.076	0.318	ALERT
Bedrooms	0.003	0.027	0.201	ALERT
City_Zone	0.001	0.027	0.206	ALERT
Bathrooms	0.001	0.024	0.104	watch
Neighborhood_Score	0.018	0.014	0.026	stable
Distance_to_Center_km	0.024	0.029	0.019	stable
Age_Years	0.011	0.015	0.006	stable
Property_Condition	0.005	0.018	0.019	stable
Has_Parking	0.000	0.000	0.000	stable

Batch 3 feature drift, drawn against the 0.20 alert line:

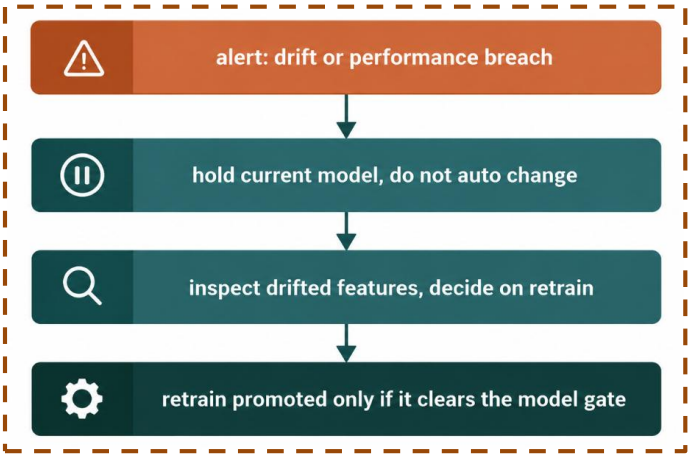


Performance drift and the alert decision

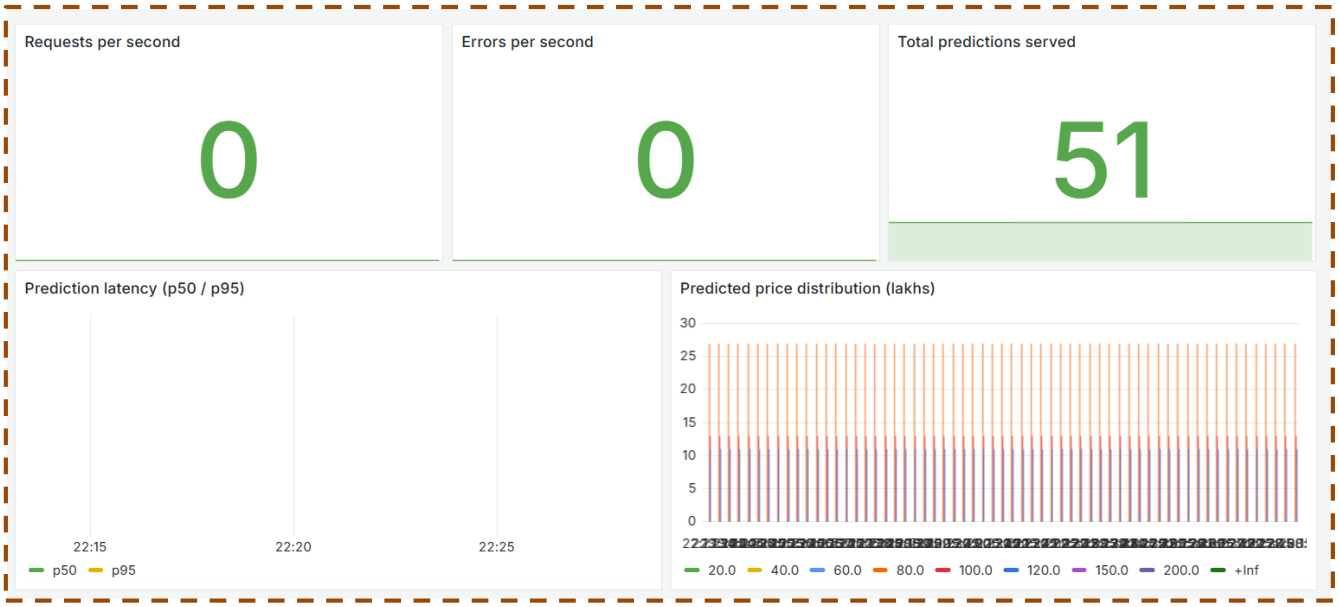
Batch	Rows	Batch RMSE	RMSE ratio	Drifted features	Alert
Batch 1	500	9.81	1.02	none	no
Batch 2	500	11.26	1.17	none	no
Batch 3	500	18.08	1.88	Area_sqft, Bedrooms, City_Zone	yes

What happens when an alert fires

An alert never changes production by itself. The response is to hold the current model, inspect the cause, and retrain only if the new model passes the model gate. This closes the loop safely.



The Grafana House Price API dashboard after sending traffic, showing request rate, error rate, latency and the predicted price distribution:



12 CI, Release and Rollback

Pull request checks

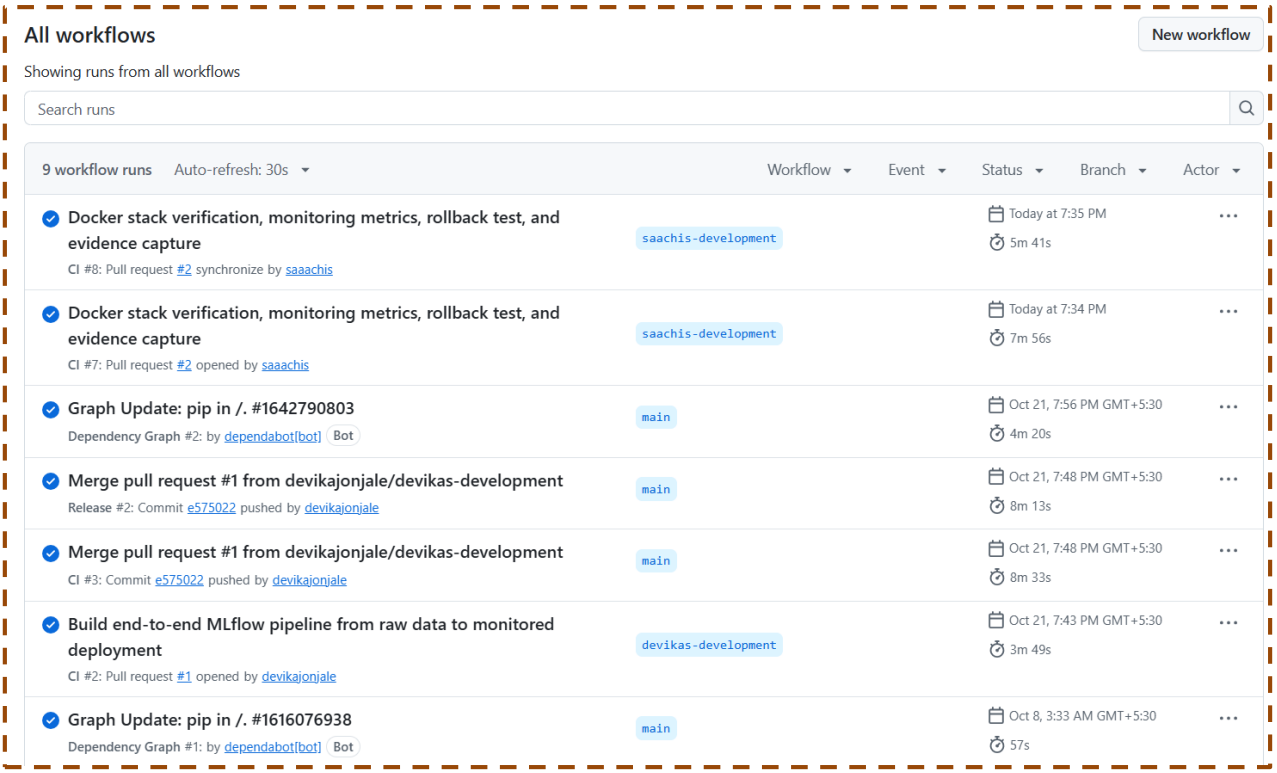
The CI workflow runs on every pull request: it installs the pinned dependencies, runs the full test suite, runs a lightweight end to end pipeline run and builds the serving image. A pull request that breaks any of these shows a red check and is blocked; a passing one shows green. Both outcomes appear in the Actions history, which is the passing and failing pull request evidence the brief asks for.

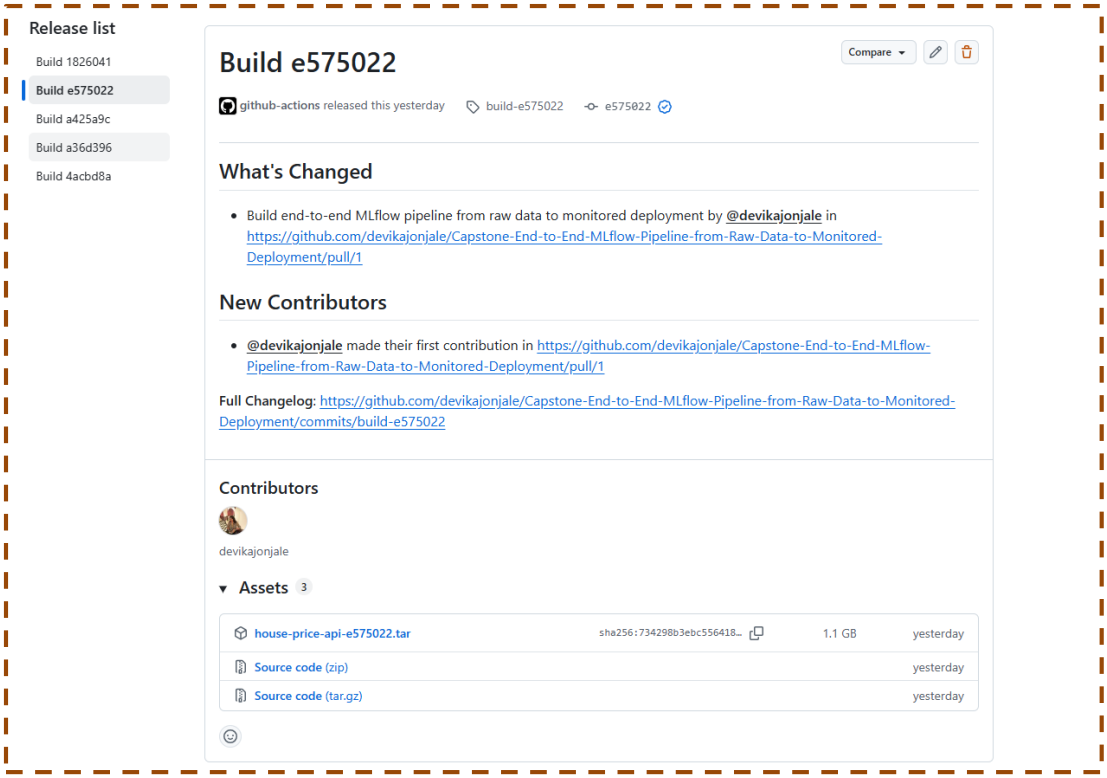
Release on merge

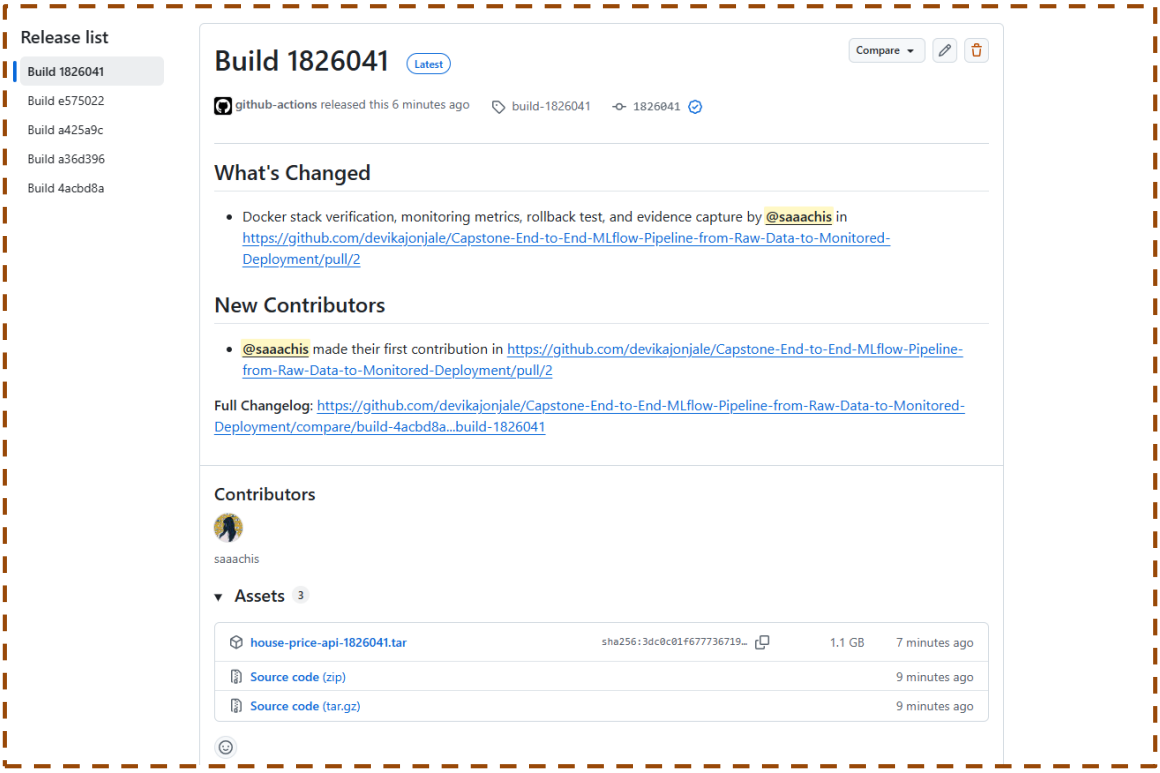
On a merge to main the release workflow builds a deployable image, tags it with the commit SHA and labels it with the same commit, and publishes a GitHub Release with the image tarball attached. Because the image carries the commit as a tag and a label, any running container can be traced back to its exact source.

Rollback

Rolling back means redeploying the image from a previous release: download the tarball from the last good release, load it into Docker, and bring the service back up. The /lineage endpoint then confirms which commit and model version is serving. Because the registry is only ever changed by the pipeline, a service rollback does not disturb the registry. The procedure is written and tested in ROLLBACK.md.







13 Containerised Deployment and Dashboards

The full stack is defined in docker-compose.yml and comes up with one command. Grafana is provisioned routinely with the Prometheus data source and the House Price API dashboard, so there is nothing to click over.

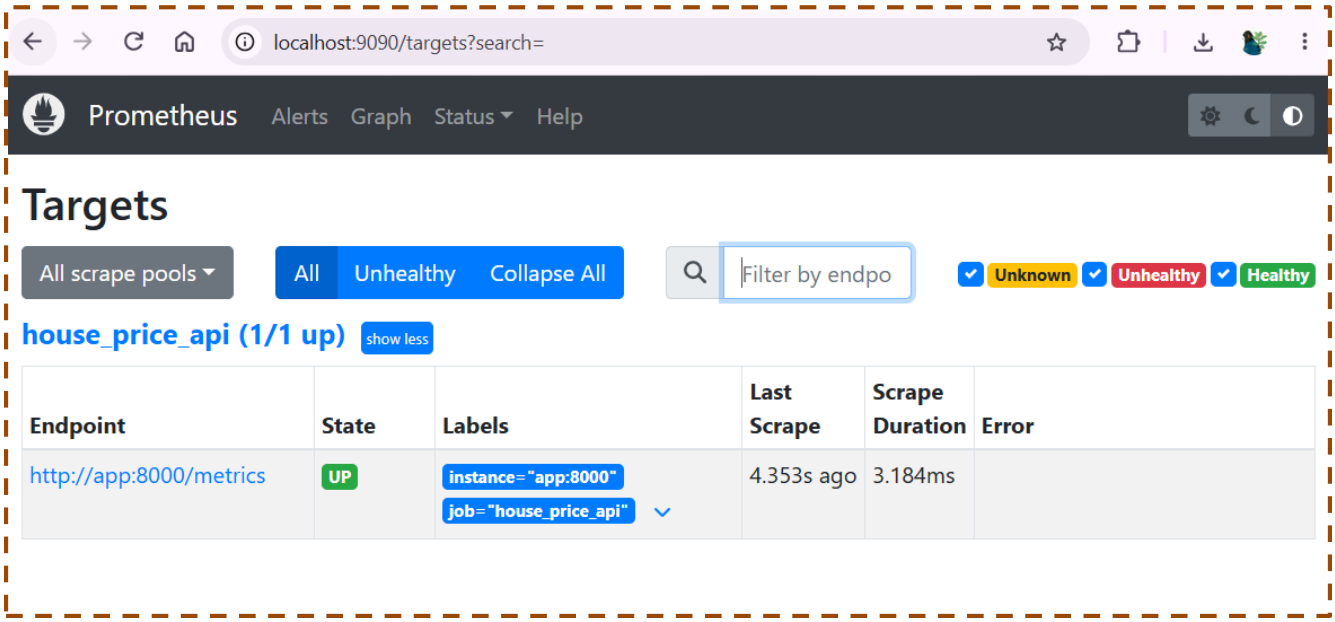
Service	Port	Purpose
app	8000	the FastAPI prediction service
prometheus	9090	scrapes the service metrics every ten seconds
grafana	3000	dashboard, admin and admin
mlflow	5000	tracking and registry user interface

How the stack connects

Prometheus scrapes the service every ten seconds; Grafana queries Prometheus. The dashboard shows request rate, error rate, latency percentiles and the predicted price distribution. Recreating the dashboard by hand and adding live alert rules are documented in GRAFANA_SETUP.md, while full container orchestration and verification procedures are maintained in the repository deployment documentation.



```
PS F:\personal-github\Capstone-End-to-End-MLflow-Pipeline-from-Raw-Data-to-Monitored-Deployment>
PS F:\personal-github\Capstone-End-to-End-MLflow-Pipeline-from-Raw-Data-to-Monitored-Deployment> docker compose ps
NAME                                                                                IMAGE                                COMMAND                  SERVICE      CREATED       STATUS       PORTS
capstone-end-to-end-mlflow-pipeline-from-raw-data-to-monitored-deployment-app-1     house-price-api:local                "uvicorn src.serving…"   app          3 hours ago   Up 3 hours   0.0.0.0:8000->8000/tcp
capstone-end-to-end-mlflow-pipeline-from-raw-data-to-monitored-deployment-grafana-1 grafana/grafana:11.2.0              "/run.sh"                grafana      3 hours ago   Up 3 hours   0.0.0.0:3000->3000/tcp
capstone-end-to-end-mlflow-pipeline-from-raw-data-to-monitored-deployment-mlflow-1  ghcr.io/mlflow/mlflow:v2.17.2        "mlflow server --hos…"   mlflow       3 hours ago   Up 3 hours   0.0.0.0:5000->5000/tcp
capstone-end-to-end-mlflow-pipeline-from-raw-data-to-monitored-deployment-prometheus-1 prom/prometheus:v2.54.1          "/bin/prometheus --c…"   prometheus   3 hours ago   Up 3 hours   0.0.0.0:9090->9090/tcp
PS F:\personal-github\Capstone-End-to-End-MLflow-Pipeline-from-Raw-Data-to-Monitored-Deployment>
```



14 Worked Example: One Full Run

This section walks through a single, complete use of the system, the way it would be used when a new batch of listings arrives. It is the story the live demonstration follows.

Step 1. Train and promote the first model

With a clean clone and the requirements installed, one command trains the model, runs both gates and promotes the result. The data gate passes on the 4,000 training rows, the model clears the quality bar, and version 1 is registered to Production.

```
$ python run_pipeline.py
... data_gate: passed ... model_gate: passed
"registered_version": 1, "status": "success"
```

Step 2. Serve a quote

The service loads the promoted pipeline and answers a quote for a three bedroom listing in the North zone.

```
$ curl -X POST localhost:8000/predict -d '{...listing...}'
{"predicted_price_lakhs": 73.42}
```

Step 3. New batches arrive and are monitored

Running monitoring over the three incoming batches produces the report below. Batches 1 and 2 pass quietly; batch 3 trips the alert.

```
$ python run_monitoring.py
| Batch   | Drifted features              | RMSE  | Alert |
| batch_1 | none                          | 9.81  | no    |
| batch_2 | none                          | 11.26 | no    |
| batch_3 | Area_sqft, Bedrooms, City_Zone| 18.08 | yes   |
```

Step 4. A bad batch is refused

If a batch arrives with impossible values, the data gate refuses it and the production model keeps serving, unchanged.

```
$ python -m scripts.demo_failures
status: rejected | data_gate: failed
production version before=1 after=1
```

Step 5. Ship a change safely

A code change opens a pull request, which CI tests and runs. On merge to main a release builds an image tagged with the commit, and if anything goes wrong the previous image is reloaded per ROLLBACK.md. At no point does anything other than the pipeline change the production model.

> **Demonstration script**
> This end to end loop, a code change through CI, a pipeline run, deployment, a monitoring update and one safe failure, is exactly the 5 to 10 minute demonstration the brief asks for.

15 Results, Contributions and Reflection

Rubric coverage

Rubric criterion	Marks	Where it is met
Problem framing and design	10	Sections 01 and 02, justified thresholds
Pipeline architecture and orchestration	20	Section 05, Prefect flows, re-runnable stages
Gates and failure handling	15	Section 08, two demonstrated failures
Tracking, registry and lineage	15	Section 09, MLflow, registry, /lineage
CI/CD and release	15	Section 12, ci.yml, release.yml, ROLLBACK.md
Deployment and monitoring	20	Sections 10, 11, 13, metrics, drift, Grafana
Demonstration and documentation	5	Section 14, README, this report

Individual contributions

Devika Jonjale (B045) was responsible for pipeline architecture, model development, automated quality gates, and lineage tracking. She designed the centralized configuration framework (config.yaml) and orchestrated the modular training and monitoring flows using Prefect.

She implemented the dual-gate governance mechanism: the Data Gate for schema and boundary validation, and the Model Gate to prevent regressions before deployment. Devika engineered the preprocessing and Random Forest training pipeline, integrated MLflow for experiment tracking and Model Registry promotion with cryptographic data versioning, built the PSI drift detection module, and developed the automated test suite ensuring deterministic, safe-failure behavior.

Saachi Shinde (B048) was responsible for containerisation, operational monitoring, disaster-recovery verification, and release engineering. She authored the multi-service Docker Compose architecture (app, prometheus, grafana, mlflow), verified live model serving endpoints (/health, /lineage, /predict, /metrics), simulated synthetic listing traffic, and validated real-time metrics flow into Prometheus and the auto-provisioned Grafana dashboard.

She executed and documented the disaster-recovery rollback procedure per ROLLBACK.md, authored and merged Pull Request #2 to trigger the automated GitHub Actions CI/CD release workflow (Build 1826041), and compiled the final technical evidence and reporting.

Reflection: what we would improve with more time

The pipeline does what the brief asks, but a few things would make it sturdier. We would add automatic retraining triggered by a drift alert, gated the same way, so the loop closes without a person in the middle. We would push serving metrics and batch drift scores into the same Grafana instance and add live alert rules rather than only the batch report. We would move the MLflow backend from a local SQLite file to a shared server so runs from CI and distributed development environments land in one place. We would widen the tests with a property based check on the feature logic and a contract test on the serving schema. Finally we would add a small, logged hyperparameter search instead of the hand chosen values.

Reproducibility

To reproduce: create a Python 3.11 virtual environment, install the pinned requirements, run pytest, then run_pipeline.py and run_monitoring.py. The docker compose stack reproduces the full serving and monitoring environment. All tools are free and open source and no secrets are stored in the repository.

Conclusion:

The Capstone End-to-End MLOps Pipeline has been fully implemented, integrated, and verified against all criteria in the project brief. From raw data ingestion, automated quality gating, and reproducible model training to MLflow lineage tracking, containerised FastAPI serving, Prometheus/Grafana real-time monitoring, and zero-downtime rollback recovery, every architectural layer operates deterministically. Automated gates prevent invalid data or sub-par models from reaching production, while GitHub Actions CI/CD ensures that every release is cryptographically tied to its source commit.

The complete system was successfully executed, stress-tested, and validated in an isolated multi-container Docker environment, providing a robust, production-grade template for reliable ML deployments.

***